Übungsblatt: Constraints

Carsten Gips (HSBI)

1 CSP.01: Logikrätsel (2P)

Betrachten Sie die Variante des berühmten "Einstein-Rätsels" auf Wikipedia.

Formulieren Sie das Problem als CSP (Variablen, Wertebereiche, Constraints) zunächst auf dem Papier. Machen Sie sich klar, was die Variablen und was deren Wertebereiche sind. Schreiben Sie die Constraints als (unäre bzw. binäre) Relationen zwischen den Variablen auf.

Hinweis: Machen Sie sich zunächst klar, was die Variablen und was deren Wertebereiche sind. Schreiben Sie die Constraints als (unäre bzw. binäre) Relationen auf.

Thema: Formulierung von Problemen als CSP

2 CSP.02: Framework für Constraint Satisfaction (3P)

Lösen Sie nun das obige Rätsel (aus CSP.01):

1. Lösen Sie das Rätsel zunächst mit dem Basis-Algorithmus `BT_Search` aus der Vorlesung.
2. Erweitern Sie den Algorithmus um die Heuristiken MRV und Gradheuristik und lösen Sie das Problem erneut. Vergleichen Sie die Ergebnisse und die Laufzeit der beiden Experimente.
3. Wenden Sie vor dem Start von `BT_Search` den AC-3 an. Erhalten Sie damit bereits eine Lösung (bzw. Unlösbarkeit)? Falls nicht, wenden Sie anschließend den ergänzten Algorithmus aus Schritt (2) an. Vergleichen Sie wieder die Ergebnisse und die Laufzeiten.
4. Wenden Sie die *Min-Conflicts* Heuristik zur Lösung des Problems an. Vergleichen Sie wieder die Ergebnisse und die Laufzeiten.

Sie können dafür eine Handsimulation anwenden oder die Algorithmen implementieren. Sie können gern auch die Java-Klassen im Paket `aima.core.search.csp` bzw. die Python-Klassen in `csp.py` als Ausgangspunkt nutzen.[1]

3 CSP.03: Kantenkonsistenz mit AC-3 (1P)

Sei $D = \{0, \ldots, 5\}$, und ein Constraintproblem definiert durch

$$\langle \{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\} \rangle$$

mit

- $c_1 = \big((v_1, v_2), \{(x, y) \in D^2 | x + y = 3\}\big)$,
- $c_2 = \big((v_2, v_3), \{(x, y) \in D^2 | x + y \leq 3\}\big)$,
- $c_3 = \big((v_1, v_3), \{(x, y) \in D^2 | x \leq y\}\big)$ und
- $c_4 = \big((v_3, v_4), \{(x, y) \in D^2 | x \neq y\}\big)$.

[1] Im Python-Code tauchen immer wieder "TODO"-Marker auf - bitte mit Vorsicht genießen!

1. Zeichnen Sie den Constraint-Graph
2. Wenden Sie den AC-3-Algorithmus auf das CSP an. Geben Sie den Zustand der Queue und das Ergebnis von `ARC_Reduce`, d.h. den Ergebniszustand des aktuellen D_i, für jede Iteration des Algorithmus an.

Thema: Handsimulation des AC-3-Algorithmus

4 CSP.04: Forward Checking und Kantenkonsistenz (1P)

Betrachten Sie erneut das CSP aus der vorigen Aufgabe und die Zuweisung $\alpha = \{v_1 \to 2\}$.

1. Erzeugen Sie Kantenkonsistenz in α. Geben Sie hierzu die Wertebereiche der Variablen vor und nach dem Erzeugen der Kantenkonsistenz an.

   Hinweis: Sie dürfen annehmen, dass der Wertebereich von Variablen mit bereits zugewiesenen Werten nur aus dem zugewiesenen Wert besteht, während unbelegte Variablen den vollen Wertebereich haben.

   Hinweis: Sie müssen zur Lösung dieser Teilaufgabe nicht den AC-3 nutze.

2. Führen Sie Forward-Checking in α aus. Vergleichen Sie das Ergebnis mit (1).

Thema: Kantenkonsistenz und Forward Checking verstehen

5 CSP.05: Planung von Indoor-Spielplätzen (3P)

Sie sind für die Planung von Indoor-Spielplätzen zuständig.

Ein Spielplatz hat eine rechteckige Form, etwa 40x100 m. Zur Vereinfachung wird diese Fläche in ein gleichmäßiges Raster unterteilt, beispielsweise 10x10 cm. Es gibt am Rand mehrere Türen (normaler Eingang, Notausgänge).

Auf dieser Grundfläche sollen verschiedene Spielgeräte angeordnet werden, beispielsweise Go-Kart-Bahnen, Hüpfburgen und Kletterberge. Diese Spielgeräte haben selbst eine rechteckige Grundfläche, wobei die Abmessungen rastergenau sein sollen (also Vielfache der Rastergröße). Weiterhin gibt es eine Bar, die als größerer rechteckiger Bereich modelliert werden kann (keine Unterscheidung in Tresen plus Tische plus Stühle o.ä. notwendig).

Die Aufgabe besteht darin, die Spielgeräte und den Bar-Bereich nach bestimmten Randbedingungen anzuordnen. Zu diesen Vorgaben gehört, dass sich die Spielgeräte nicht überlappen oder berühren, dass bestimmte Sicherheitsabstände eingehalten werden (1m zw. den Spielgeräten), dass Entspannungszonen eingerichtet werden und dass Sichtlinien gewährleistet sind (z.B. sollten die Aktivitäten am Kletterberg vom Barbereich aus beobachtet werden können). Die Bar ist bevorzugt direkt am Eingang zu finden. Notausgänge dürfen nicht verstellt werden.

Abstrahieren Sie das gegebene Problem angemessen und geben Sie eine geeignete Modellierung als CSP an. Definieren Sie sich ein paar Spielgeräte und lösen Sie das Problem mit Hilfe von *MAC* und *Min-Conflicts*.

Thema: Modellierung eines Real-World-Problems

6 Bonus: DSL für Constraint Solving: MiniZinc (2P)

MiniZinc ist eine *Domain Specific Language* (DSL) zum Modellieren von Constraint- und Optimierungs-Problemen.

Schauen Sie sich das Tutorial an. Stellen Sie das Einstein-Rätsel oder das Spielplatz-Problem in MiniZinc dar. Nutzen Sie zur Ausführung einen der im Playground angebotenen Solver. Vergleichen Sie die Modellierung und die Laufzeiten mit Ihren Ergebnissen aus den vorigen Aufgaben.

Thema: DSL zur Formulierung von Constraint-Problemen

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ae651b8 (homework: fix typo (CSP), 2025-10-04)